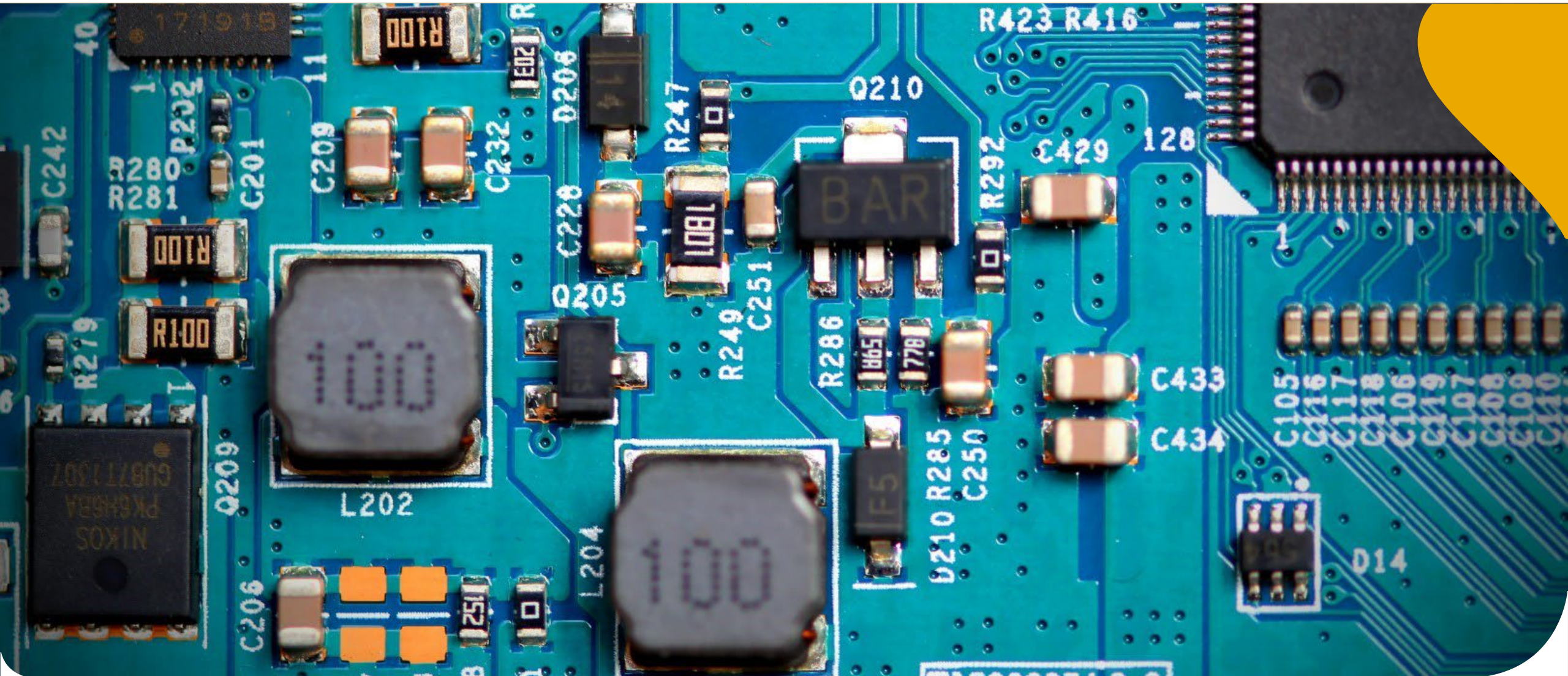


# Design(E)-314

## Lecture 1





# Lecturers

**Dr Armand  
du Plessis**

**Prof Herman  
Engelbrecht**





# Who we are

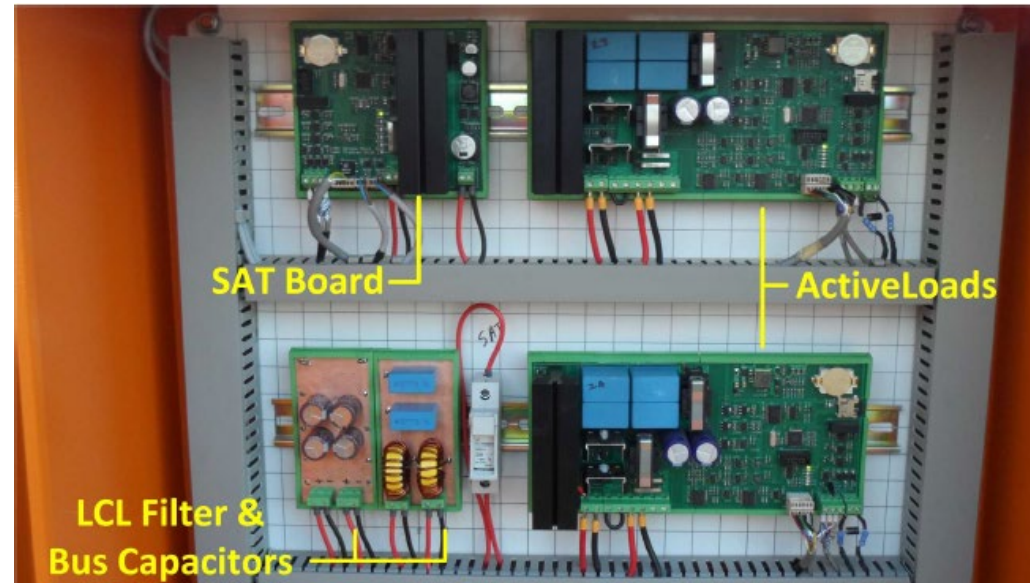


**Stellenbosch**  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

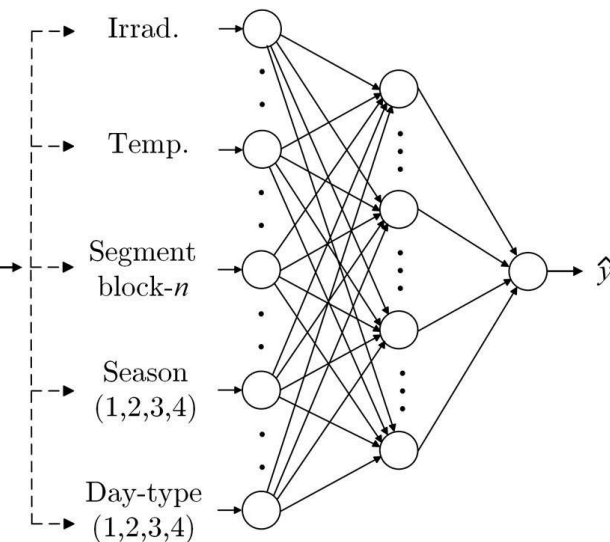
forward together  
sonke siya phambili  
saam vorentoe

## Dr Armand du Plessis

- BEng, MEng, PhD – Stellenbosch
- Masters in solar PV system performance monitoring
- PhD in solar power forecasting
- Teaching:
  - Electrotechnique 143
  - Design E314
- Research:
  - Energy forecasting and electricity markets



Time-series  
data set



# Who we are



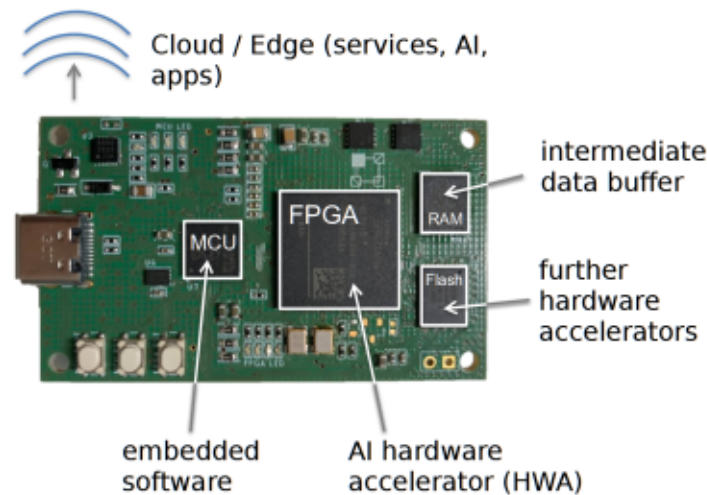
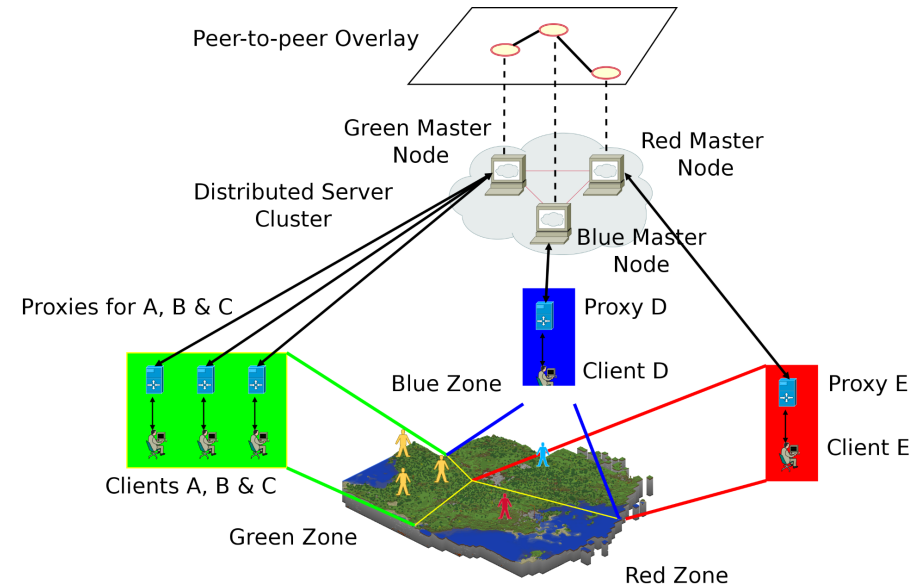
Stellenbosch

UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

## Prof Herman Engelbrecht

- Masters in speech recognition of South African phonemes
- PhD in efficient decoding of hidden Markov models
- Teaching:
  - Design E314
  - Computer Programming 143
- Research:
  - Scalable Virtual Environments,
  - Distributed Systems, Embedded AI,
  - Reinforcement Learning

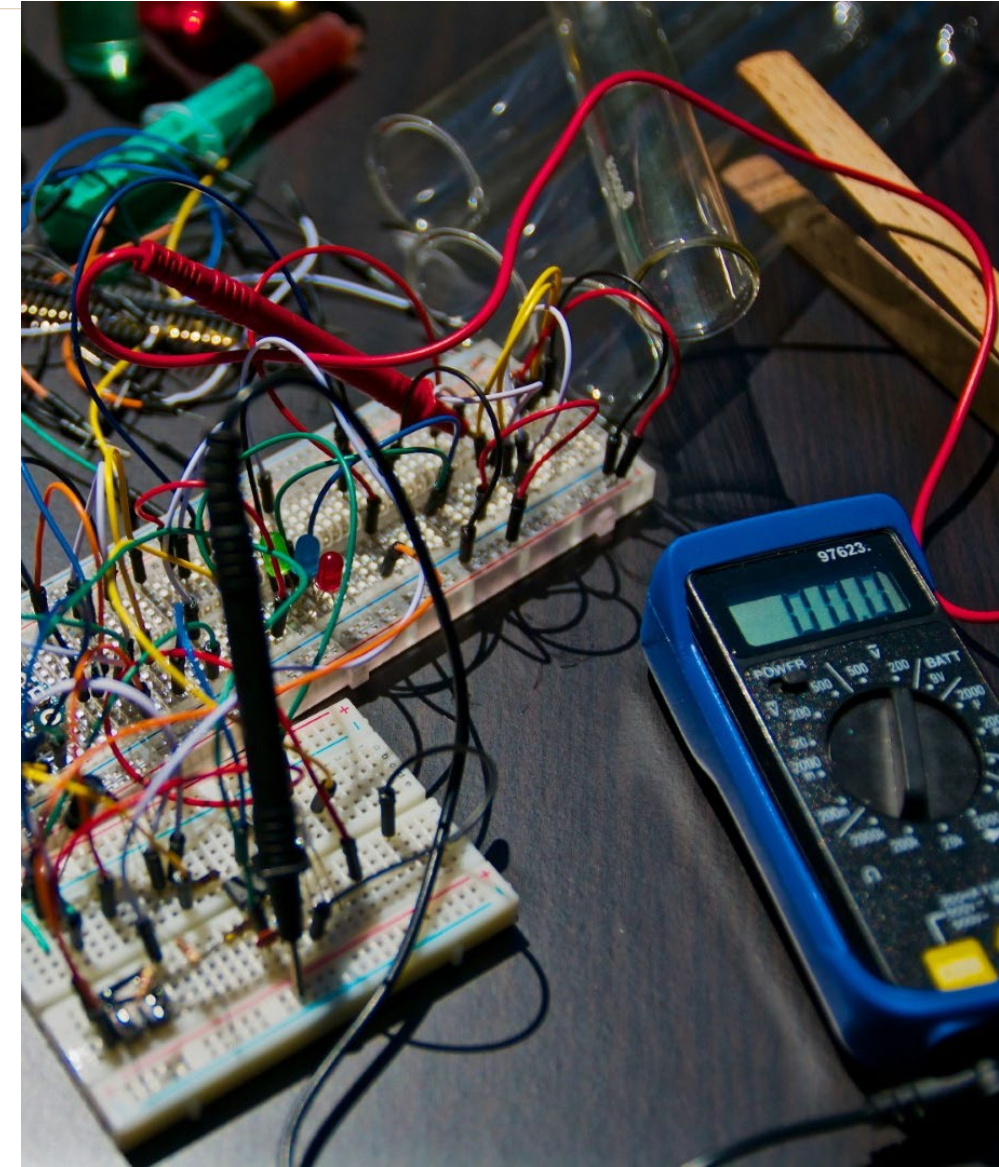




# Overview

## Demo 1 cycle

- Module framework
- Lectures
- Rules: Sub-minimums
- Demos
- PDD – Project Definition Document
- Practical 1
- Design Cycle
- Project for 2026



## Module framework

- Is available in English and Afrikaans on STEM Learn (previously known as SUN Learn)
- Contains all the official rules, assessment definition, schedule
  - Homework: Read it carefully

## Lectures

- Tuesdays: 08:10 – 09:00 M&M
- Tuesdays: 09:10 – 10:00 E&E (all focus areas) and Comp. Science

## Practicals

- Thursdays: 09:00 – 12:00 M&M and Comp. Science
- Fridays : 10:00 – 13:00 E&E (all focus areas)

## Module Rules to take note of (see module framework)

- E-Design final mark has 3x items
  - Demos
  - Tests
  - Report
- Each student completes their own circuit board, report and tests
  - No group work marking
  - You are assessed individually
- **50%** Subminimum for all 3 parts
- **Pass:** mark  $\geq$  50% for each of the items

## Module Rules to take note of (see module framework)

### Examples:

T = 65.7%

D = 80%

R = 80%

=> PM = 75% (distinction)

T = 50%

D = 80%

R = 80%

=> PM = 65% (pass, limited by  $PM_{max}$ )

T = 48.6%

D = 80%

R = 80%

=> PM = 45% (fail, sub-minimum not achieved)

### Graduate Attributes

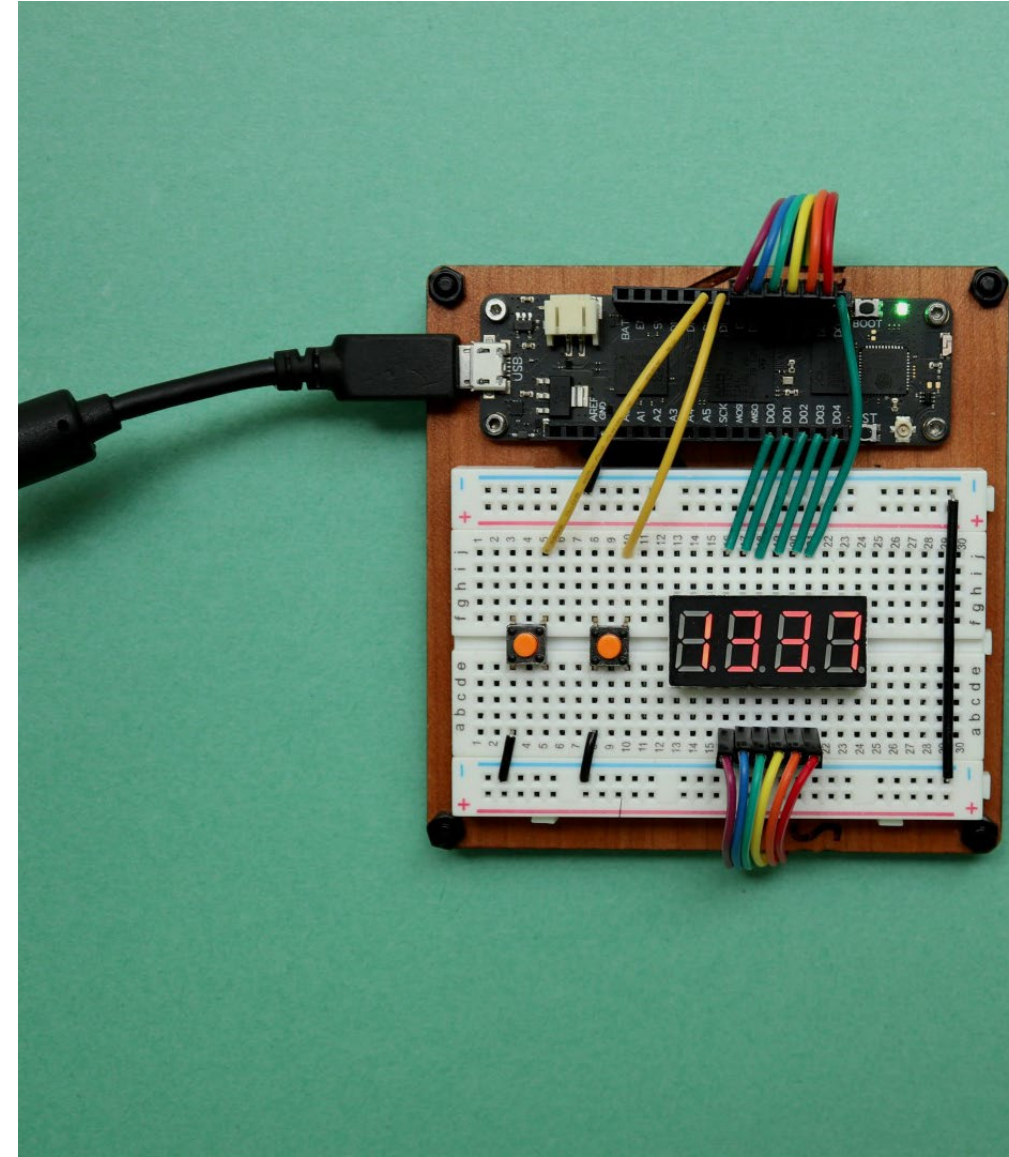
- GAs are strictly examined
- Failure to satisfy ANY of the GAs will result in failing the module



# Demonstrations

## Demo 1: 26 & 27 February

- Demos are used to test and confirm system functionality
- New features are added for each demo cycle
  - 4x demo cycles on total
- Thursdays & Fridays during practical session
- You **must** Demo on the day allocated to you
  - Thursdays: 09:00 – 12:00 M&M
  - Fridays : 10:00 – 13:00 E&E



## How to prepare

- Get the functionality working as per the PDD
- Extensive TESTING on your workbench
  - ↳ Keep detailed recording of testing (pictures & methodology documentation)

## Test station system

- Automated + Manual marks
- Based on rubrics provided in PDD
- Demo query feedback via email – 7 day query limit

## GIT → Learn how to use it and USE IT

- Used to do version control of your software

- Have it up to date BEFORE you demo

*You can get  
80% of the  
work done in 20% of the time*

**NB**

### Pro-Tip:

- 80/20 rule → Demo 1 & 2 aim for 100%. Demo 3 & 4 aim for 80%.
- Do not try to get 100% - at the expense of your other modules





# PDD: Project Definition Document



Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

Document >> specifies project/device requirements

- Will be updated as required – always use the latest version
  - In real life project scope is updated all the time
  - The same applies here – you must keep up with document scope → *It's a living document*
  - New changes will be indicated on PDD front page
- Found on course website on STEM Learn



## Pro-Tip:

- For each Demo, read through the PDD
- List/summarise all design requirements (make it your own)



## Importance

- Only element that gets externally moderated
- Is the culmination of your hard work
-  GAs are evaluated in the reports

## Continuous writing

- Work on the report(s) from day one
- Work on it continuously
- Concepts => Draft => Proof copy

## Preliminary report

- To give experience and prepare for final report
- Peer reviewed with extensive rubric
- Peer review quality checked with random sample – heavy penalty for non-compliance to rubric

## Final report

- Hand-in Monday after classes ends
- Full rubric will be available
- Details to follow in PDD



NOTA BENE \*

## Pro-Tip:

- After your Demo
  - Immediately write up all of the designs and implementations
  - Take measurements
  - Do it while it is still fresh in your mind
- 
- The final report writing should be you “putting together” all of your writing into a single document – it should not be the first time you write about your design.



# Design cycle



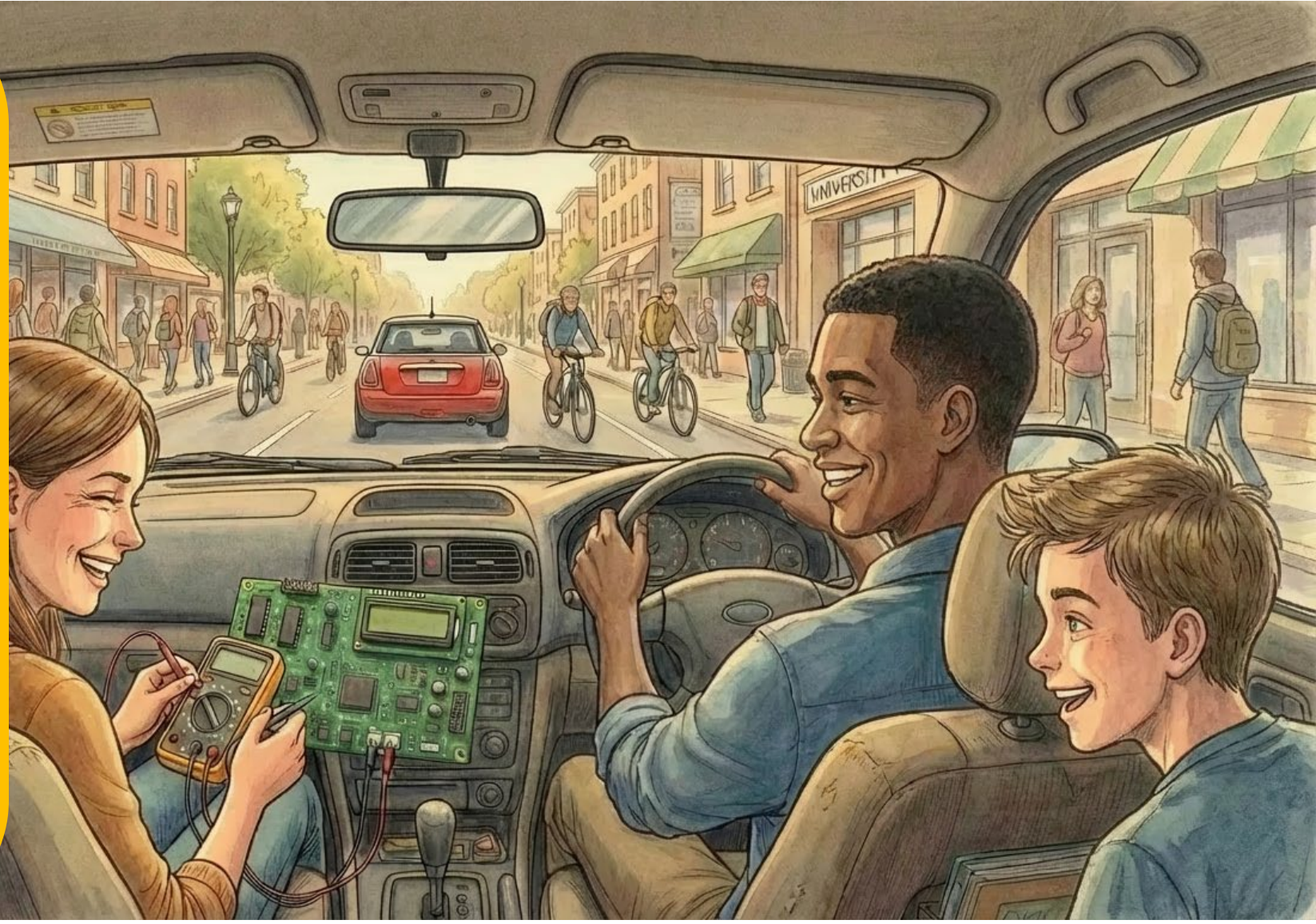
## Example

- Build a temperature sensor.
- What sensors can work? What design will work.
- Choose best candidate sensor (datasheet). Simulate?
- Build on breadboard or PCB. Create schematics, PCB layout
- Test against specifications using measurement equipment – Multimeter/Scope etc.
- Report on your system performance/limitations



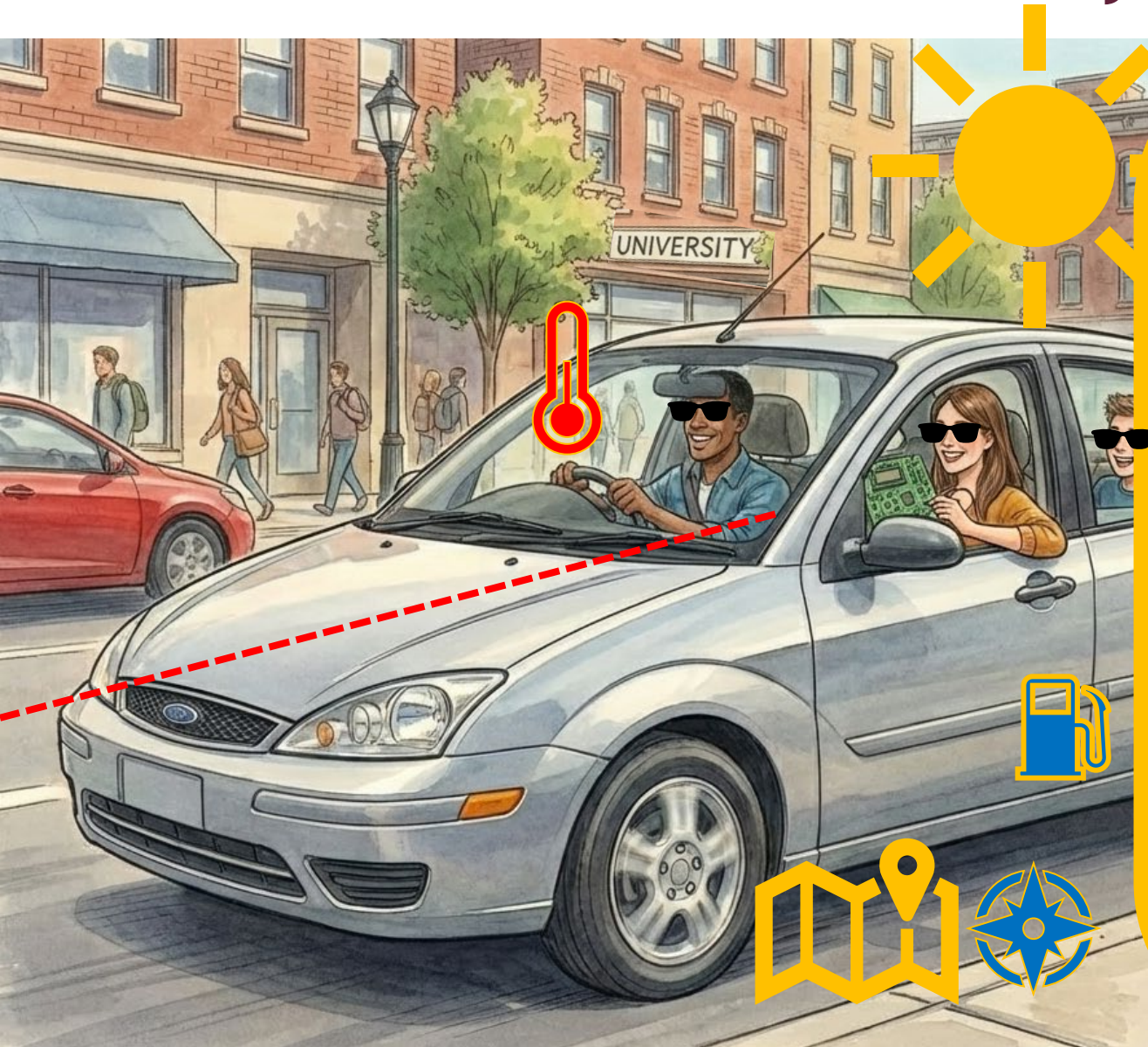
# Project Description: Embedded - Driver Assistance System (E-DAS)

- Most student cars on campus are “entry level” and slightly older (5+ years).
- In Stellenbosch, many engineering students would love to have some form of **driver assistance** and **analytics** in their cars.





# Project Description: Embedded - Driver Assistance System (E-DAS)



As an engineer, your goal is to develop an embedded system that you can use to:

- Detect **distance** relative to other vehicles/objects
- Determine low **light** and high ambient **temperature** conditions
- Log fuel consumption and determine **efficiency** stats
- Capture **driving** (acceleration) and ~~GPS~~ (location) data for analytics

# Practical 1 - Demo 1 cycle

- Get your components at the Lab Managers at the start of the first practical
- Important components for Demo 1
  1. STM32 microcontroller
  2. IDE
  3. PCB (Student Board)
  4. Power Supply
  5. LEDs
  6. USART communication

*\*Temperature sensor is most important for Demo 1*

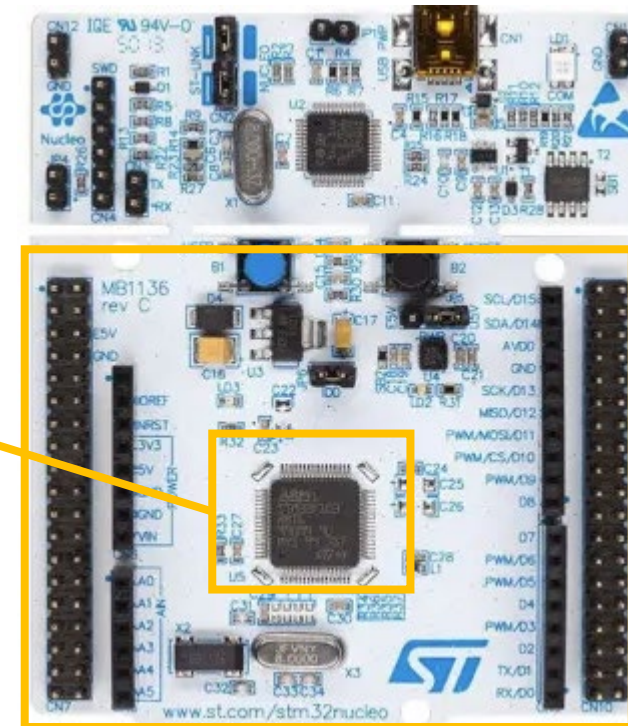
# Practical 1 - Demo 1 cycle STM32 microcontroller

## STM32 Nucleo Development board

- We use the: STMicroelectronics STM32 Nucleo-64  
MCU Development Board: **NUCLEO-F411RE**

### Microcontroller Unit

- MCU: STM32F411RET6
- Core: Arm Cortex-M4 with FPU (Floating Point Unit)
- Max Clock Speed: 100 MHz
- Flash Memory (Storage): 512 Kbytes
- ADC Specs: 1× 12-bit ADC
- USART, SPI (5 interfaces), I2C (3 interfaces)



Motherboard: MB1136



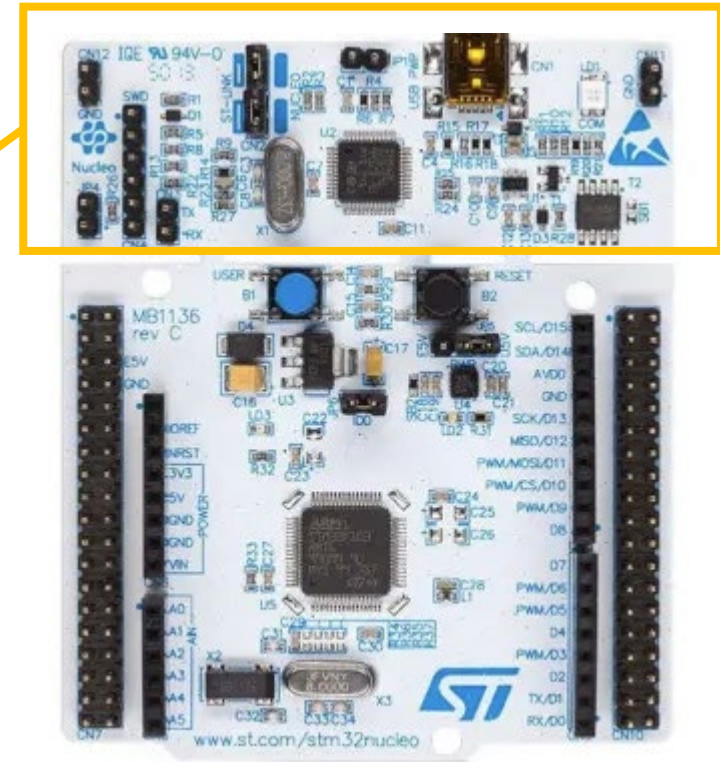
# Practical 1 - Demo 1 cycle STM32 microcontroller

ST-LINK/V2-1 is the integrated hardware tool.

Acts as the bridge between your PC and the STM32 microcontroller.

**Function:** It loads your compiled code into the STM32's flash memory.

**Debugging:** It allows you to pause code, step through lines, and inspect variables live in your IDE (STM32CubeIDE)



# Practical 1 - Demo 1 cycle

## STM32 microcontroller



Stellenbosch

UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

## STM32 Nucleo MCU Development board: Datasheets & Docs

| Document Name                     | Summary and Usage                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RM0383 Reference Manual</b>    | Provides application developers with complete info. on how to use the memory and peripherals of the <b>STM32F411xC/E microcontrollers</b> . This document details <b>technical descriptions</b> of peripheral functionalities, register maps, and overall system architecture. <b>Use</b> when you need to configure the external hardware peripherals such as specific comms. interfaces (I2C, USART, SPI) or hardware controllers like the DMA and ADC.          |
| <b>ES0287 Errata Sheet</b>        | This document describes known device limitations (hardware deviations) and documentation errata for the STM32F411xC/E microcontrollers. Use it to troubleshoot unexpected behaviour and identify available software workarounds for specific peripheral issues.                                                                                                                                                                                                    |
| <b>STM32 Nucleo-64 Data Brief</b> | This brief summarizes the common features and ordering information for various STM32 Nucleo-64 development boards. Consult this for a high-level comparison of board variants and to decode product order codes.                                                                                                                                                                                                                                                   |
| <b>PM0214 Programming Manual</b>  | This manual provides a full description of the STM32 Cortex-M4 processor's programming model, its complete instruction set, and core peripherals like the NVIC and MPU. Reference this for low-level <b>software development</b> , exception handling details, and core-specific configurations.                                                                                                                                                                   |
| <b>TN1235 Technical Note</b>      | This note provides an overview of all ST-LINK debugger/programmer probe versions, including standalone probes and those embedded in development boards. Use it to understand the differences between ST-LINK generations (V1, V2, V3) and to interpret the "COM" LED status indicators.                                                                                                                                                                            |
| <b>UM1724 User Manual</b>         | This manual details the hardware layout, mechanical dimensions, and default configurations for the <b>MB1136</b> reference board used in many Nucleo-64 kits. <b>Use</b> it to find specific jumper settings for power selection, solder bridge configurations, and full pinout maps for Arduino and ST morpho connectors. (p.33)                                                                                                                                  |
| <b>UM1727 User Manual</b>         | This document provides a step-by-step "Getting Started" guide for beginners to build and debug their first applications using various supported IDEs like EWARM and MDK-ARM. Use this as a tutorial for setting up system requirements, installing drivers, and loading firmware packages.                                                                                                                                                                         |
| <b>STM32F411RE Datasheet</b>      | Provides technical specifications, pinout descriptions, memory maps, and detailed <b>electrical characteristics</b> for these microcontrollers. It should be used during hardware design to verify absolute maximum ratings and operating conditions. Additionally, it serves as the primary reference for mapping peripheral functions to pins. <b>Use</b> the datasheet during hardware design to ensure the device is electrically compatible with your system. |

# Practical 1 - Demo 1 cycle

## Integrated Development Environment (IDE)



Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

### STM32 Nucleo MCU Development board

- Integrated development environment (IDE) from STMicroelectronics is for developing, configuring, and debugging applications on STM32 microcontrollers.
- Download and use **V1.19.0** to ensure compatibility with Lab PCs

Download here: <https://www.st.com/en/development-tools/stm32cubeide.html>

### Integrated Development Environment for STM32

Get Software

Download databrief

Overview

Documentation

Tools & Software

### Product overview

Description

All features

Get Software

Featured Products

Featured Videos

Recommended for you

### Get Software

| Part Number                        | General Description                  | Latest version | Download                   | All versions                   |
|------------------------------------|--------------------------------------|----------------|----------------------------|--------------------------------|
| <a href="#">+</a> STM32CubeIDE-DEB | STM32CubeIDE Debian Linux Installer  | 2.0.0          | <a href="#">Get latest</a> | <a href="#">Select version</a> |
| <a href="#">+</a> STM32CubeIDE-Lnx | STM32CubeIDE Generic Linux Installer | 2.0.0          | <a href="#">Get latest</a> | <a href="#">Select version</a> |
| <a href="#">+</a> STM32CubeIDE-Mac | STM32CubeIDE macOS Installer         | 2.0.0          | <a href="#">Get latest</a> | <a href="#">Select version</a> |
| <a href="#">+</a> STM32CubeIDE-RPM | STM32CubeIDE RPM Linux Installer     | 2.0.0          | <a href="#">Get latest</a> | <a href="#">Select version</a> |
| <a href="#">+</a> STM32CubeIDE-Win | STM32CubeIDE Windows Installer       | 2.0.0          | <a href="#">Get latest</a> | <a href="#">Select version</a> |

STMicroelectronics recommends always keeping your software up to date

2.0.0

1.19.0

1.18.1



# Practical 1 - Demo 1 cycle Integrated Development Environment (IDE)

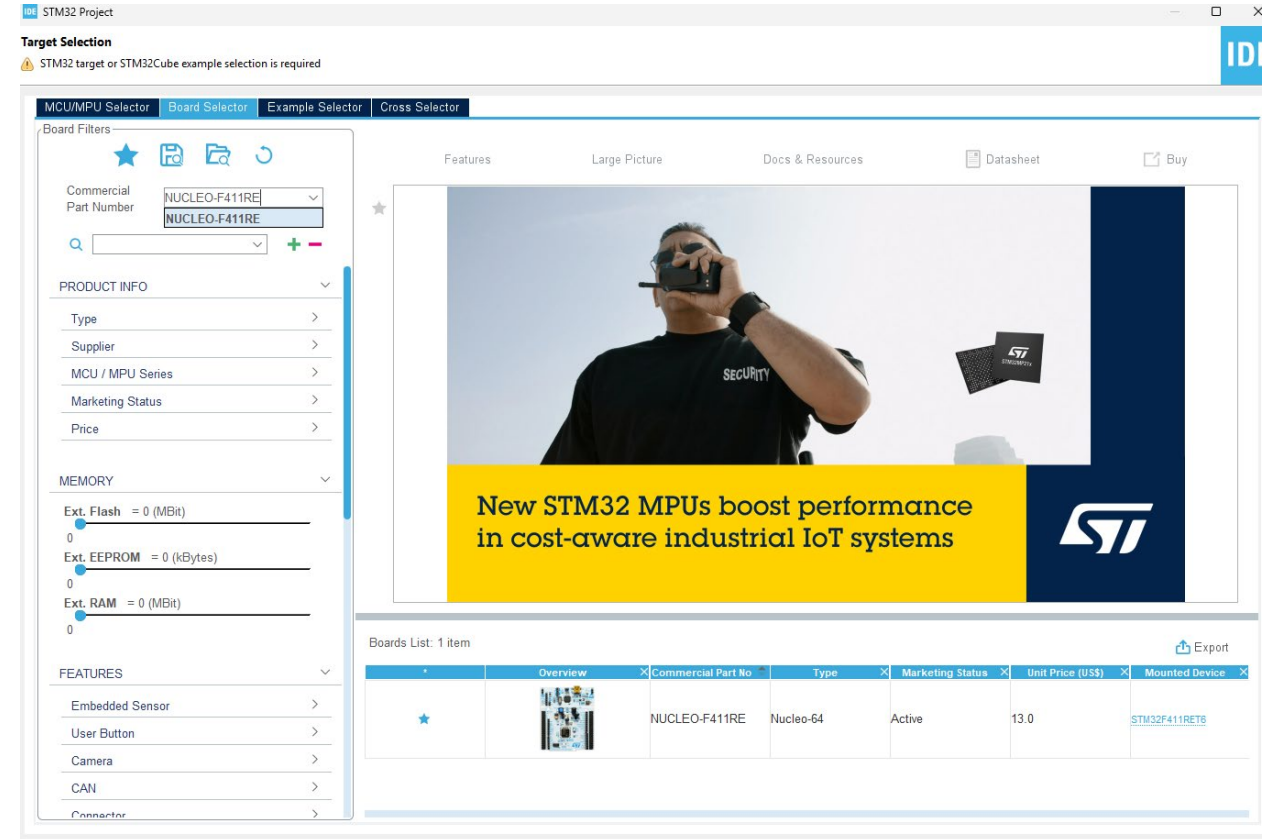


Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

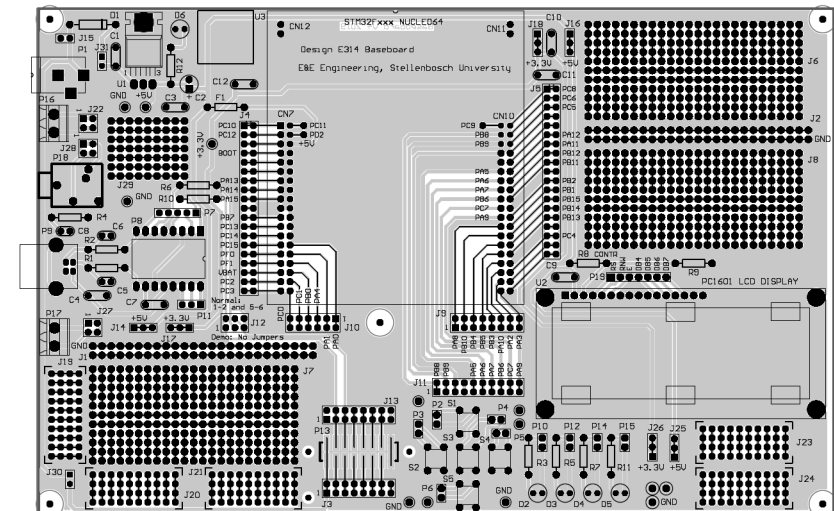
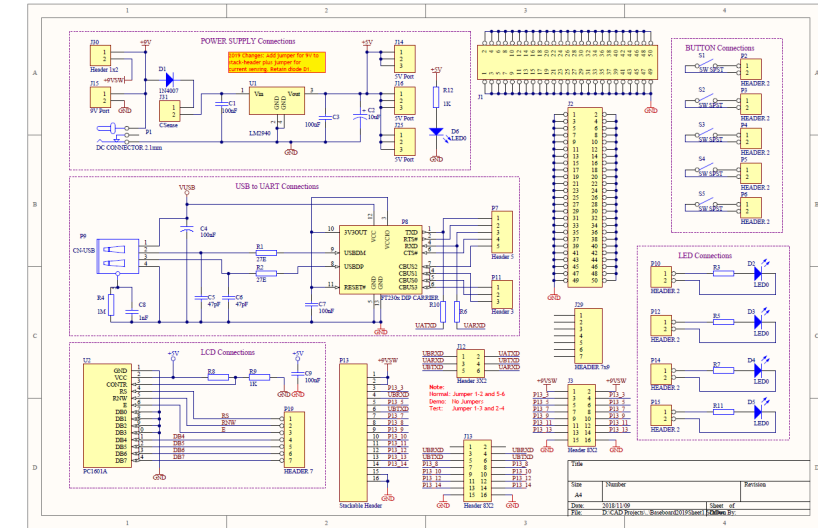
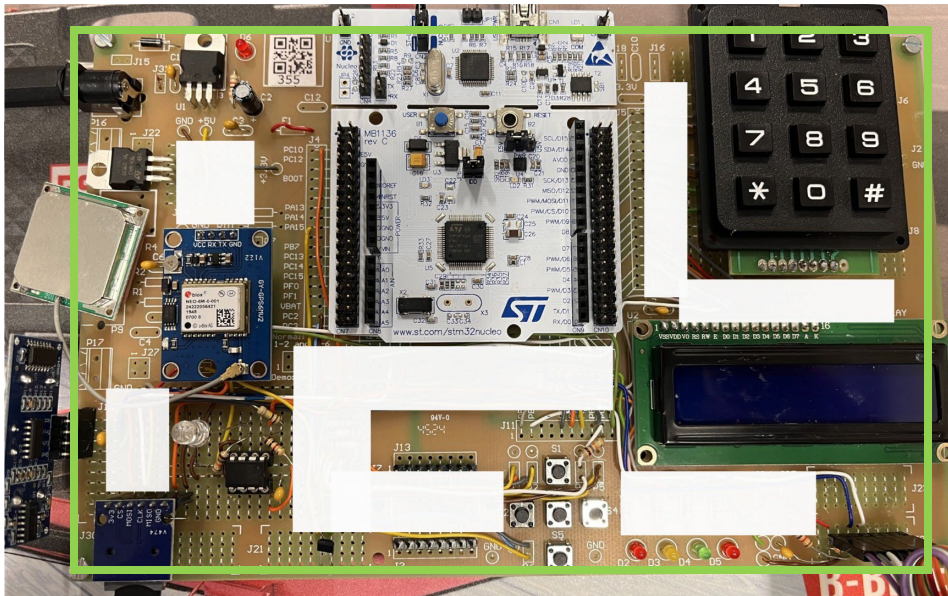
## STM32 Nucleo MCU Development board

1. To get started: Look at **STEMLearn**  
resource: “STM32CubeIDE Getting Started Guide”
2. Download the STM32CubeIDE Install file  
for Windows (Version 1.19.0)
3. **Note:** “Board Selector” setup, do not use  
the MCU/MPU Selector



# Practical 1 - Demo 1 cycle PCB Student Board

- PCB (Printed Circuit Board) – Datasheet schematics provided
- This is the “mother board” that **you** get to configure for your own unique functionality.



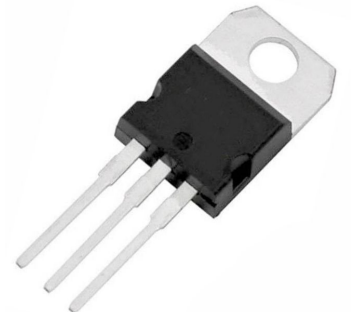
# Practical 1 - Demo 1 cycle Power Supply

## Main Power Supply

- Vehicle 12V (lighter socket) >> But can the STM32 Nucleo Dev. Board handle 12V input?  
Q for STM32F411RE Datasheet
- What is the voltage rating of the General-Purpose Input/Output (GPIO) pin?  
↳ Pg. 38 ⇒ The voltage rating is 3.3V

## 5V Regulator

- We need a 5V (to power STM32 Nucleo Board) and 3.3V power supply (to power peripherals and comms)



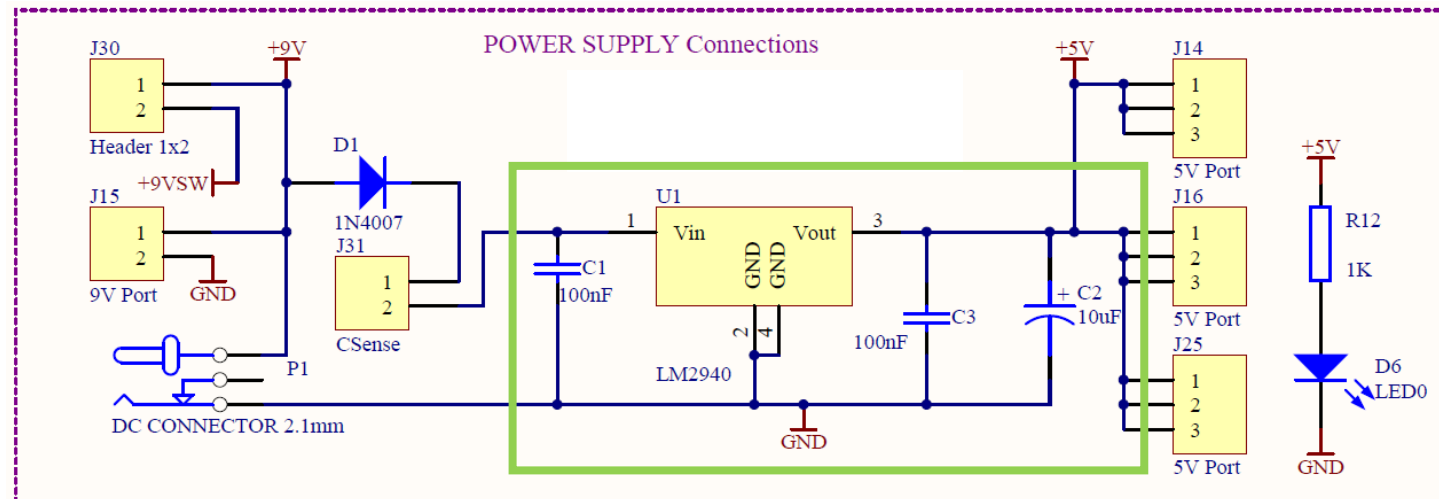


# Practical 1 - Demo 1 cycle Power Supply

## 5V Regulator

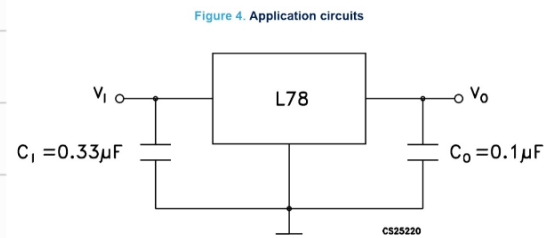
- See datasheet Figure 4 - Application circuit
- Design provided in PCB schematic file

- Is a heat-sink necessary?  *$V_i$  is within specified limits  
As long as current is  $< 1A$ , heat sink is not necessary*



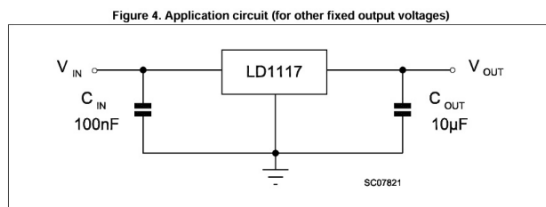
## APPLICATION CIRCUIT FOR L78C05 (5V) REGULATOR

Found on pg 4 of datasheet



## APPLICATION CIRCUIT FOR LD1117 (3.3V) REGULATOR

Found on pg 8 of datasheet

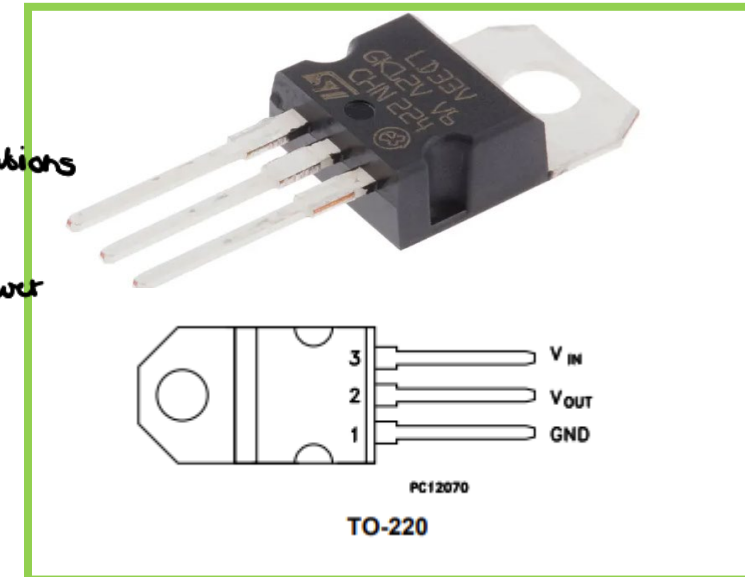


# Practical 1 - Demo 1 cycle Power Supply

## 3.3V Regulator

- You need to design circuit and build yourself
- What is a decoupling/bypass capacitor?
  - A small capacitor placed bwn power line & GND to "clean up" the power supply
    - smooths out fluctuations
    - Eliminates high f noise
    - Protects IC from unstable power
- What is dropout voltage?  $1V_{Typ}$ 
  - The minimum  $\Delta$  bwn  $V_{in}$  &  $V_{out}$  at which the regulator can still maintain the correct output  $V$
- What are the electrical characteristics of this

regulator? (V, A, Watt)





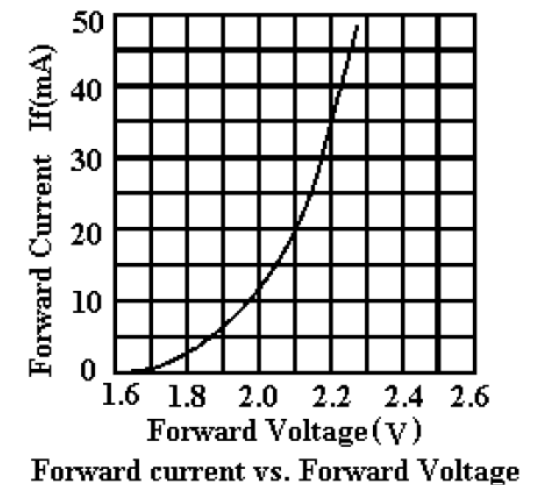
# Practical 1 - Demo 1 cycle LEDs



Stellenbosch  
UNIVERSITY  
IYUNESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

- Indicator LEDs (see PDD for functionality)
- Your design needs to consider:
  - Total board peripheral consumption to be limited to maximum current draw capability of 5V and 3.3V regulators.
  - GPIO pins serve as power supply. How will you protect the GPIO pins from exceeding output current limit?
  - What is the Forward voltage drop over the LED? How does this affect your design?



# Practical 1 - Demo 1 cycle UART Comms



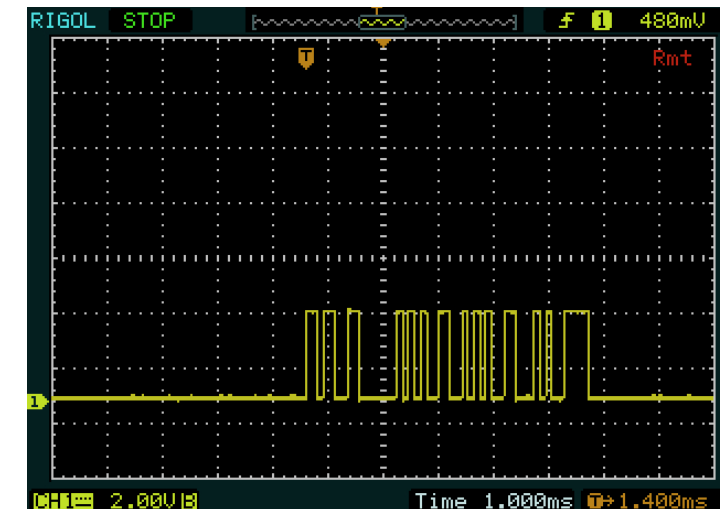
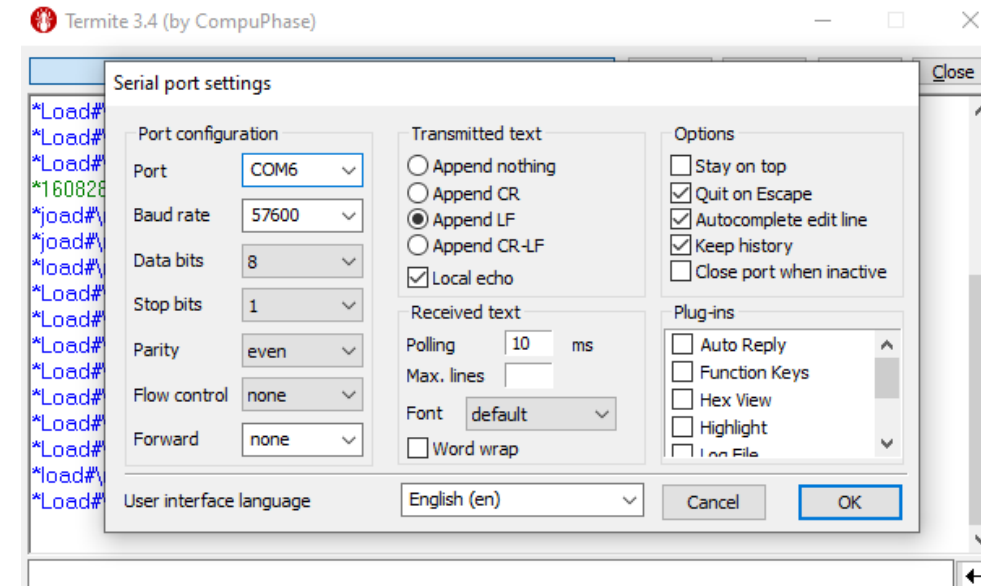
Stellenbosch  
UNIVERSITY  
IYUNIVESITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

- For UART comms with PC/Laptop use terminal program
  - Options: TerraTerm, Termite, RealTerm, Putty, CoolTerm
- Measure timing with oscilloscope (RX and TX lines) to ensure you are within timing requirements.

## Pro-Tip:

When using an oscilloscope, always use the oscilloscope probes (do not use the normal 'crocodile clip' cables)



# Practical 1 - Demo 1 cycle

## UART Comms



Stellenbosch  
UNIVERSITY  
IYUNISITHI  
UNIVERSITEIT

forward together  
sonke siya phambili  
saam vorentoe

- UART (Universal Asynchronous Receiver/Transmitter)
  - 2 Wires (TX, RX)
  - UART: Devices have their own internal clocks. You must configure them to the exact same speed (e.g., 9600 baud). If one device is 2% faster than the other, the data becomes corrupted.
  - STM32 has a USART, which we operate in *asynchronous mode* (acting just like a UART) to talk to your PC via USB, or the test station (which is another STM32 board)
  - Use Termite

In 99% of your projects, you will configure the USART peripheral to run in Asynchronous mode. When you do this, it behaves exactly like a UART. We effectively 'downgrade' the fancy USART to a simple UART because it saves us a wire (no clock line needed)."





# Questions?

